

**Università Politecnica delle Marche**

**Facoltà di Ingegneria**

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione

---



# **ChatBot DataScience**

**Sviluppo di un assistente conversazionale tramite il  
Framework RASA**

**Candidato**

Erxhes Dedja — 1119284

**Docenti**

Prof. Domenico Ursino

Dott. Christopher Buratti

---

**Anno Accademico 2025-2026**

# Indice

---

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Framework Rasa</b>                                         | <b>1</b>  |
| 1.1      | Chatbot . . . . .                                             | 1         |
| 1.2      | Rasa . . . . .                                                | 2         |
| 1.3      | Rasa NLU . . . . .                                            | 2         |
| 1.4      | Rasa Core . . . . .                                           | 3         |
| <b>2</b> | <b>Configurazione e Architettura del Progetto</b>             | <b>4</b>  |
| 2.1      | Introduzione . . . . .                                        | 4         |
| 2.2      | Ambiente Virtuale . . . . .                                   | 4         |
| 2.3      | Database dei Prodotti dello Store . . . . .                   | 5         |
| 2.4      | Struttura del Progetto . . . . .                              | 6         |
| 2.5      | Logica Applicativa e Azioni Personalizzate . . . . .          | 6         |
| 2.5.1    | Algoritmo di Ricerca e Risoluzione dei Nomi . . . . .         | 7         |
| 2.5.2    | Gestione del Dialogo e Suggerimenti Contestuali . . . . .     | 7         |
| <b>3</b> | <b>Funzionalità del Sistema</b>                               | <b>8</b>  |
| 3.1      | Introduzione . . . . .                                        | 8         |
| 3.2      | Avvio della Conversazione . . . . .                           | 8         |
| 3.3      | Informazioni sullo Store . . . . .                            | 8         |
| 3.4      | Esplorazione dei Prodotti in Catalogo . . . . .               | 8         |
| 3.5      | Interrogazione sui Prodotti . . . . .                         | 8         |
| 3.6      | Riconoscimento Robusto tramite Fuzzy Matching . . . . .       | 9         |
| 3.7      | Suggerimenti Contestuali e Persistenza del Contesto . . . . . | 9         |
| 3.8      | Esempio di Conversazione Completa . . . . .                   | 9         |
| <b>4</b> | <b>Integrazione e Canale Telegram</b>                         | <b>11</b> |
| 4.1      | Creazione del Bot Telegram . . . . .                          | 11        |
| 4.2      | Architettura di Collegamento e Instradamento . . . . .        | 12        |
| 4.3      | Configurazione del Server Flask . . . . .                     | 12        |
| 4.4      | Avvio dei Servizi . . . . .                                   | 12        |
| 4.5      | Validazione Sperimentale delle Interazioni . . . . .          | 13        |
|          | <b>Bibliografia</b>                                           | <b>17</b> |

# Elenco delle figure

---

|     |                                                                                      |    |
|-----|--------------------------------------------------------------------------------------|----|
| 1.1 | Logo ufficiale del framework Rasa. . . . .                                           | 2  |
| 2.1 | Struttura del progetto Rasa personalizzata. . . . .                                  | 6  |
| 3.1 | Esempio di dialogo completo: disponibilità, prezzo e colore. . . . .                 | 10 |
| 4.1 | Procedura di generazione del bot tramite l'interfaccia di BotFather. . . . .         | 11 |
| 4.2 | Inizializzazione del tunnel ngrok e generazione dell'endpoint pubblico. . . . .      | 13 |
| 4.3 | Bootstrap del server Flask e notifica di avvenuta registrazione del webhook. . . . . | 13 |
| 4.4 | Interrogazione dello stato di disponibilità di un articolo tramite Telegram. . . . . | 14 |
| 4.5 | Risoluzione del prezzo di listino associato al prodotto in esame. . . . .            | 14 |
| 4.6 | Estrazione delle varianti di taglia e colore memorizzate nel database. . . . .       | 15 |
| 4.7 | Risposta del sistema per un articolo non disponibile in magazzino. . . . .           | 15 |
| 4.8 | Visualizzazione dell'elenco prodotti filtrato per target demografico. . . . .        | 16 |

# Capitolo 1

## Framework Rasa

---

### 1.1 Chatbot

---

Un **chatbot** è un sistema che simula conversazioni umane, consentendo agli utenti di interagire con i dispositivi digitali in modo naturale e intuitivo. A seconda dell'architettura, può basarsi su regole predefinite o sull'intelligenza artificiale per rispondere in modo flessibile e contestuale.

Le principali tipologie di chatbot sono:

- **Chatbot basati su regole:** Spesso chiamati clickbot, questi sistemi seguono un percorso di conversazione già stabilito, proponendo all'utente opzioni pronte per guidare il dialogo. Non sono in grado, però, di gestire domande o richieste fuori programma.

Vengono utilizzati per gestire FAQ, prenotazioni automatiche e supporto tecnico di primo livello, fornendo risposte rapide. Sono ideali per attività con interazioni prevedibili e ripetitive, come il servizio clienti e le vendite online, ottimizzando l'efficienza operativa.

- **Chatbot basati su intelligenza artificiale:** Questi sistemi sono più avanzati: sfruttano l'apprendimento automatico (machine learning) e la comprensione del linguaggio naturale (NLP) per capire e rispondere a richieste espresse a parole nostre. Rispetto ai modelli a regole fisse, riescono a gestire conversazioni complesse e ad adattarsi a situazioni diverse.

La loro caratteristica principale è che imparano e migliorano con il tempo. Grazie alle interazioni con le persone e all'aggiunta di nuove informazioni, diventano sempre più precisi ed efficaci. Per questo motivo sono ideali per molti ambiti, dall'assistenza clienti fino al supporto tecnico più specializzato.

## 1.2 Rasa

---

Rasa è un framework *open-source* per creare chatbot e assistenti virtuali. Si distingue perché riesce a gestire dialoghi complessi e a personalizzare le risposte, sfruttando l'intelligenza artificiale per comprendere al meglio gli utenti.

A differenza di molti servizi commerciali già pronti, che spesso sono rigidi e limitati, Rasa offre agli sviluppatori il controllo totale e la massima flessibilità su come il chatbot elabora le informazioni. In questo modo è possibile costruire un assistente su misura per qualsiasi esigenza. L'architettura di Rasa è composta da due moduli principali:

- **NLU** (Natural Language Understanding);
- **Core**.

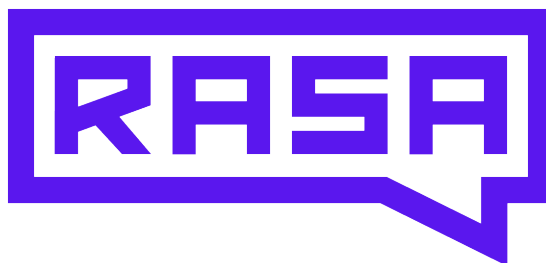


Figura 1.1: Logo ufficiale del framework Rasa.

## 1.3 Rasa NLU

---

Rasa NLU (Natural Language Understanding) è il modulo che si occupa di comprendere e interpretare il linguaggio naturale. Il suo compito principale è analizzare il testo inserito dall'utente per identificare l'**intento** (l'obiettivo della richiesta) e estrarre le **entità** (i parametri specifici, come nomi di prodotti, taglie, date o località).

In particolare, Rasa NLU utilizza modelli di *machine learning* per:

- Classificare il testo in base agli intenti;
- Identificare le entità, cioè i dettagli chiave della frase.

La classificazione e l'estrazione avvengono tramite modelli statistici addestrati su dataset di esempi reali. Questo approccio permette a Rasa NLU di riconoscere le diverse varianti linguistiche con cui gli utenti esprimono la stessa intenzione, migliorando così precisione e affidabilità del sistema.

## 1.4 Rasa Core

---

Rasa Core è il modulo che gestisce l'andamento e la storia della conversazione. Mentre Rasa NLU si limita a capire il significato dei singoli messaggi, Rasa Core decide come portare avanti il dialogo. Per farlo, utilizza delle logiche personalizzabili (chiamate policy, come TEDPolicy, RulePolicy e MemoizationPolicy; ricorda che, dalla versione 3.x, il sistema non usa più direttamente l'apprendimento per rinforzo).

Ogni volta che l'utente scrive qualcosa, Rasa Core analizza tutto quello che è stato detto fino a quel momento e sceglie l'azione o la risposta più adatta. Questa capacità si basa su esempi di dialoghi reali (chiamati stories), che insegnano al chatbot come comportarsi in modo logico e coerente nelle varie situazioni.

## Capitolo 2

# Configurazione e Architettura del Progetto

---

### 2.1 Introduzione

---

Per assicurare il corretto funzionamento di Rasa e evitare conflitti con le librerie Python di sistema, è stato creato un ambiente virtuale dedicato. In questo modo, l'ambiente di sviluppo è isolato, evitando interferenze con altre versioni di moduli e semplificando la gestione delle dipendenze richieste dal framework.

### 2.2 Ambiente Virtuale

---

Per inizializzare l'ambiente virtuale e configurare il progetto, sono necessari i seguenti componenti:

- **Python 3.10:** Versione di riferimento stabile per Rasa (release 3.4.x e successive).
- **Anaconda o Miniconda:** Strumenti per gestire pacchetti e ambienti virtuali.
- **Visual Studio Code (VSCode):** Ambiente di sviluppo integrato (IDE) per scrivere il codice e gli script delle azioni.

Dopo aver installato Miniconda, configurare l'ambiente virtuale seguendo questi passaggi:

1. **Aprire** Anaconda Prompt o il terminale di comando del sistema.
2. **Creare** un nuovo ambiente virtuale specificando Python 3.10:

```
1 conda create --name rasa_env python=3.10
```

3. **Attivare** l'ambiente virtuale appena creato:

```
1 conda activate rasa_env
```

4. **Installare** Rasa e le dipendenze necessarie per il pre-processing e l'integrazione del server:

```
1 pip install rasa pandas thefuzz python-Levenshtein flask
```

## 2.3 Database dei Prodotti dello Store

Ci basiamo sul dataset **AC Milan Store Products**, che contiene le informazioni strutturate sugli articoli dello store ufficiale dell'AC Milan. Ogni riga del file descrive un prodotto, con le proprietà elencate nella Tabella 2.1.

| Colonna       | Descrizione                                                                                                                      |
|---------------|----------------------------------------------------------------------------------------------------------------------------------|
| Categoria     | Indica la macro-categoria di appartenenza del prodotto (es. <i>Kit Da Gara, Allenamento</i> ).                                   |
| Tipologia     | Specifica il tipo di articolo (es. <i>Maglia, Pantaloncini, Giacca, Pallone</i> ).                                               |
| Nome          | Denominazione ufficiale completa del prodotto, inclusa la stagione di riferimento (es. <i>Milan Home 2024/25</i> ).              |
| Disponibilità | Indica se il prodotto è disponibile in magazzino ( <i>Sì</i> o <i>No</i> ).                                                      |
| Sesso         | Specifica il target demografico di riferimento: <i>Uomo, Donna, Bambino, Unisex</i> .                                            |
| Taglie        | Elenco delle taglie disponibili (es. <i>XS, S, M, L, XL, XXL</i> per adulti o <i>6 anni, 8 anni, ecc.</i> per la linea bambino). |
| Colore        | Colore principale del prodotto (es. <i>Rosso e Nero</i> ).                                                                       |
| Prezzo        | Prezzo di listino in euro.                                                                                                       |

Tabella 2.1: Schema del database dei prodotti.

Le azioni personalizzate interagiscono con i dati usando la libreria **pandas**, che gestisce le operazioni di pulizia, uniformazione e normalizzazione del testo.



## 2.4 Struttura del Progetto

Dopo aver inizializzato il workspace con il comando `rasa init`, la struttura di base è stata modificata per integrare le funzionalità specifiche del progetto. La struttura finale è mostrata nella Figura 2.1.

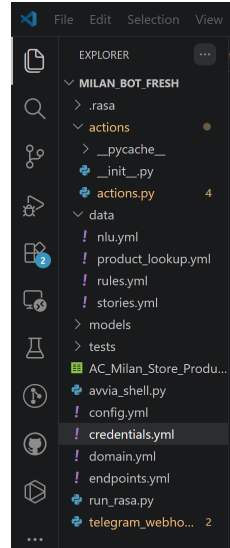


Figura 2.1: Struttura del progetto Rasa personalizzata.

I file principali del sistema sono: `actions/actions.py` (contiene le azioni personalizzate per cercare i prodotti, controllare la disponibilità, verificare i prezzi e applicare filtri), `data/nlu.yml` (esempi pratici per insegnare al chatbot a capire cosa desidera l'utente), `data/rules.yml` (regole fisse e automatiche per gestire risposte immediate), `data/stories.yml` (esempi di dialoghi reali a più battute per allenare il chatbot), `data/product_lookup.yml` (lista di riferimento con i nomi precisi dei prodotti), `config.yml` (impostazioni generali su come analizzare i messaggi e gestire i dialoghi), `domain.yml` (file di configurazione principale con l'elenco di ciò che il chatbot può capire, ricordare e rispondere), `credentials.yml` (chiavi di accesso per collegare Telegram e altri canali esterni), `endpoints.yml` (parametri per far comunicare le diverse parti del sistema), `telegram_webhook_server.py` (script Flask che fa da tramite tra Telegram e Rasa), `AC_Milan_Store_Products.csv` (il catalogo completo dei prodotti in formato CSV).

## 2.5 Logica Applicativa e Azioni Personalizzate

Il dialogo con l'utente e la consultazione del catalogo sono gestiti da azioni personalizzate che lavorano direttamente sul file CSV. Quando l'utente scrive un messaggio, Rasa NLU ne comprende l'intenzione ed estrae le informazioni chiave (come il nome del prodotto e il destinatario, ad esempio uomo, donna o bambino). Subito dopo, Rasa Core avvia l'azione adatta, che cerca i dati nel file e risponde all'utente in modo semplice e naturale.

### 2.5.1 Algoritmo di Ricerca e Risoluzione dei Nomi

Per gestire errori di digitazione e varianti nei nomi dei prodotti, la ricerca nel catalogo segue una strategia a tre livelli:

1. **Corrispondenza esatta:** Verifica se la stringa inserita corrisponde esattamente al nome ufficiale nel database (ignorando maiuscole/minuscole).
2. **Corrispondenza parziale:** Controlla se l'input è una parte del nome ufficiale (o viceversa).
3. **Fuzzy Matching:** Se i controlli precedenti falliscono, il sistema usa un algoritmo basato sulla libreria `thefuzz` (successore di `fuzzywuzzy`) per trovare somiglianze tra stringhe. Se la somiglianza supera una soglia dell'85%, il sistema collega l'input al prodotto corretto (es. "Pallone Milan" → "Pallone Milan Essential").

### 2.5.2 Gestione del Dialogo e Suggerimenti Contestuali

Oltre a mostrare i dati richiesti, le azioni personalizzate rendono la conversazione più fluida grazie ad alcune funzioni utili:

- **Disponibilità:** controlla se il prodotto è in magazzino, filtrando anche in base al destinatario (uomo, donna, ecc.) se l'utente lo specifica.
- **Prezzo:** indica il costo del prodotto, adattando la risposta anche per una variante specifica.
- **Taglie e colori:** mostra l'elenco delle taglie disponibili e le varianti di colore in cui è acquistabile l'articolo.
- **Reset dei filtri:** azzera le preferenze di ricerca dopo aver mostrato i risultati, così da poter iniziare una nuova ricerca da zero.

Per facilitare il dialogo, il chatbot suggerisce sempre come continuare la conversazione senza che l'utente debba ripetere ogni volta il nome del prodotto (ad esempio: "Vuoi sapere anche il prezzo o le taglie disponibili? Scrivi `prezzo` o `taglie`").

## Capitolo 3

# Funzionalità del Sistema

---

### 3.1 Introduzione

---

In questo capitolo descriviamo le funzionalità del chatbot, spiegando come interagisce con l'utente e come risponde per migliorare l'esperienza nello store dell'AC Milan. Il sistema è progettato per rispondere rapidamente a domande su catalogo, disponibilità, prezzi, taglie e colori, permettendo ricerche filtrate per categoria e target (uomo/donna/bambino).

### 3.2 Avvio della Conversazione

---

L'utente può iniziare la conversazione con un saluto (es. "Ciao" o "Buongiorno") o chiedendo direttamente informazioni. Dopo aver ricevuto il primo input, il chatbot si presenta e aspetta le prossime domande.

### 3.3 Informazioni sullo Store

---

Il chatbot può fornire una panoramica completa sull'offerta dello store. L'utente può chiedere al chatbot le categorie principali dei prodotti o una descrizione generale dei servizi disponibili.

### 3.4 Esplorazione dei Prodotti in Catalogo

---

Una funzionalità chiave del sistema è la visualizzazione dinamica degli articoli in magazzino. Durante la conversazione, l'utente può vedere tutto il catalogo o filtrarlo per tipo di prodotto (es. "mostrami le maglie") o target (es. "cosa avete per bambino?").

### 3.5 Interrogazione sui Prodotti

---

Il sistema permette di ottenere informazioni dettagliate su ogni prodotto. Il backend analizza la richiesta e estrae disponibilità, prezzo, taglie e colore, applicando anche il filtro

demografico.

Le principali operazioni sono:

- **Disponibilità:** Permette di controllare se un prodotto è disponibile in magazzino. Il sistema risponde in modo preciso, applicando il filtro demografico se specificato (es. “La maglia Milan Home 2024/25 da uomo è disponibile?”).
- **Prezzo:** Mostra il prezzo di listino del prodotto. Il chatbot restituisce il prezzo, adattando la risposta se la richiesta è per una variante specifica (es. uomo/donna).
- **Taglie:** Per abbigliamento (maglie, pantaloncini, giacche), l’utente può chiedere le taglie disponibili. Il sistema mostra l’elenco delle taglie, filtrato per sesso o fascia d’età.
- **Colore:** Mostra il colore principale del prodotto.
- **Filtri combinati:** Permette di esplorare il catalogo combinando target (uomo/donna/bambino) e categoria (es. Kit Da Gara, Allenamento), senza dover ricordare i nomi esatti dei prodotti.

### 3.6 Riconoscimento Robusto tramite Fuzzy Matching

---

Grazie al fuzzy matching, il sistema riesce a riconoscere i prodotti anche con errori di digitazione, refusi o nomi incompleti. Ad esempio, se l’utente scrive “Pallone Milan” (invece del nome completo), il sistema lo collega automaticamente a “Pallone Milan Essential”, permettendo di mostrare tutte le informazioni sul prodotto. Questa flessibilità rende l’interazione tollerante e adatta a scenari reali.

### 3.7 Suggerimenti Contestuali e Persistenza del Contesto

---

Dopo ogni risposta, il chatbot suggerisce all’utente come continuare la conversazione, ricordando il prodotto già menzionato per evitare ripetizioni (es. “Ti dico anche il prezzo o le taglie? Scrivi `prezzo` o `taglie`.”).

### 3.8 Esempio di Conversazione Completa

---

La Figura 3.1 mostra un esempio reale di interazione con il chatbot. Nell’esempio, l’utente chiede inizialmente la disponibilità della maglia “Milan Home 2024/25 da uomo”, poi ne interroga il prezzo e il colore, senza dover ripetere il nome del prodotto grazie ai suggerimenti contestuali.

Come si vede nel dialogo, il chatbot risponde in modo coerente, guida l’utente tra le diverse informazioni e gestisce correttamente una conversazione a più turni. Tutte le risposte vengono generate dinamicamente dai dati del file CSV e dalle azioni personalizzate implementate nel sistema.

```
Bot loaded. Type a message and press enter (use '/stop' to exit):
Your input -> Ciao
👋 Ciao! Benvenuto nello store ufficiale del Milan. Come posso aiutarti?
Your input -> La maglia Milan Home 2024/25 da uomo è disponibile?
✅ Il prodotto *'Milan Home 2024/25'* è disponibile! (per uomo)
Ti dico anche il prezzo o le taglie? Scrivi 'prezzo' o 'taglie'.
Your input -> Prezzo
💰 Il prezzo di *'Milan Home 2024/25'* è **95.00€**.
Vuoi conoscere le taglie? Scrivi 'taglie'.
Your input -> Di che colore?
🎨 Il colore di *'Milan Home 2024/25'* è: **Rosso e Nero**.
Posso aiutarti con disponibilità, prezzo o taglie.
Your input ->
```

Figura 3.1: Esempio di dialogo completo: disponibilità, prezzo e colore.

## Capitolo 4

# Integrazione e Canale Telegram

L'ultimo passo del progetto è integrare il chatbot con Telegram, per renderlo accessibile tramite un'interfaccia mobile moderna e reattiva. Per esporre il server locale su Internet è stato usato **ngrok**. L'approccio standard di Rasa (webhook diretto con Telegram) aveva problemi su Windows a causa della gestione asincrona dell'*event loop*. Abbiamo quindi sviluppato un'architettura alternativa con un server Flask come intermediario.

### 4.1 Creazione del Bot Telegram

Per creare il bot su Telegram: accedere a Telegram e contattare **BotFather**, creare un nuovo bot con il comando `/newbot`, assegnare un nome (es. `MilanStoreBot`) e un username (es. `@MilanOfficialStoreBot`), generare e salvare il token di autenticazione per accedere alle API di Telegram.

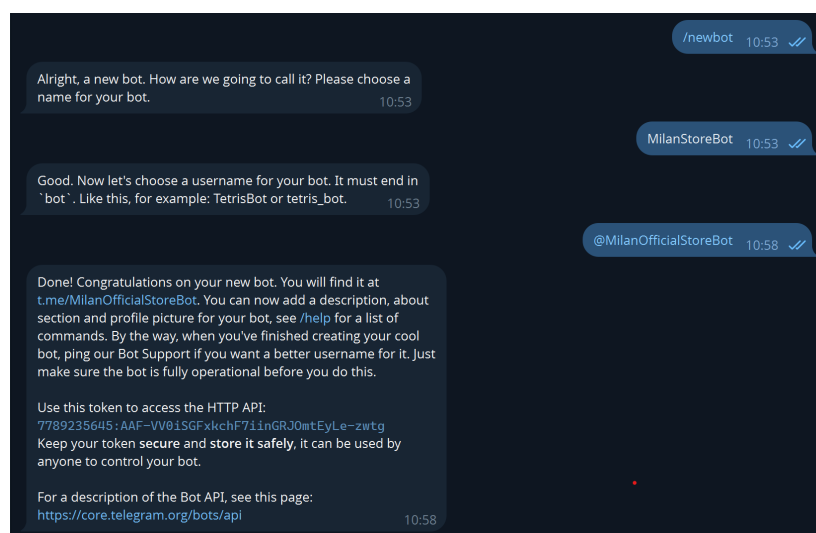


Figura 4.1: Procedura di generazione del bot tramite l'interfaccia di BotFather.

## 4.2 Architettura di Collegamento e Instradamento

Per risolvere i problemi di sincronizzazione su Windows, abbiamo sviluppato un webhook personalizzato in Flask che funge da intermediario tra Telegram e Rasa. Il flusso dei messaggi è il seguente: l'utente invia un messaggio su Telegram, **ngrok** trasmette la richiesta al server Flask locale (porta 5000), Flask analizza la richiesta e la inoltra a Rasa tramite API REST (porta 5005), Rasa elabora il messaggio e invia le risposte a Flask, Flask riceve le risposte e le inoltra all'utente su Telegram.

Questa soluzione ha mantenuto tutte le funzionalità richieste (**ngrok**, webhook, canali esterni) garantendo la stabilità del sistema su Windows.

## 4.3 Configurazione del Server Flask

Lo script `telegram_webhook_server.py` contiene la logica del server e registra automaticamente l'URL del webhook su Telegram all'avvio.

```
1 TELEGRAM_TOKEN = "YOUR_TOKEN_HERE"
2 RASA_URL = "http://localhost:5005/webhooks/rest/webhook"
3 webhook_url = "https://<your-ngrok-url>/webhook"
```

Listing 4.1: Estratto del server webhook Flask.

I token di autenticazione di Telegram sono anche nel file `credentials.yml` di Rasa, necessario per attivare il canale REST:

```
1 telegram:
2   access_token: "YOUR_TELEGRAM_TOKEN_HERE"
3   verify: "MilanOfficialStoreBot"
4
5 rest:
6   # Abilitazione del canale REST per il middleware Flask
```

Listing 4.2: Configurazione del canale di comunicazione in `credentials.yml`.

## 4.4 Avvio dei Servizi

Per mettere online il chatbot su Telegram, occorre aprire quattro finestre del terminale nella cartella del progetto, attivando in ciascuna l'ambiente virtuale `rasa_env`. I comandi da avviare sono: `rasa run actions` (porta 5055) per il server delle azioni, `rasa run -enable-api -cors "*" (porta 5005)` per il server Rasa, `python telegram_webhook_server.py` (porta 5000) per avviare lo script Flask di gestione, e infine `ngrok http 5000` per ricevere i messaggi da internet. Quest'ultimo comando genera un indirizzo web pubblico temporaneo (es. `https://<url-ngrok>`) che permette a Telegram di inviare i messaggi direttamente al nostro server Flask.

```

ngrok - tunnel local ports to public URLs and inspect traffic

USAGE:
  ngrok [command] [flags]

COMMANDS:
  api          CLI to api.ngrok.com
  completion   generates shell completion code for bash or zsh
  config       update or migrate ngrok's configuration file
  credits      prints author and licensing information
  help         help about any command
  http         start an HTTP tunnel
  service      run and control ngrok as a background service
  start        start endpoints in the config file by name
  tcp          start a TCP tunnel
  tls          start a TLS endpoint
  update       update ngrok to the latest version
ngrok

♦ One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             jasmathers21@gmail.com (Plan: Free)
Version             3.39.1-msix-stable
Region              Europe (eu)
Latency             31ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://departed-urgent-borrowing.ngrok-free.dev -> http://localhost:5000

Connections
  ttl   opn   rt1   rt5   p50   p90
   0     0    0.00  0.00  0.00  0.00

```

Figura 4.2: Inizializzazione del tunnel ngrok e generazione dell'endpoint pubblico.

```

Microsoft Windows [Versione 10.0.26200.8246]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Windows\System32>cd C:\Users\MrWhi\milan_bot_fresh

C:\Users\MrWhi\milan_bot_fresh>C:\Users\MrWhi\miniconda3\Scripts\activate.bat rasa_env

(rasa_env) C:\Users\MrWhi\milan_bot_fresh>python telegram_webhook_server.py
Set webhook response: {'ok': True, 'result': True, 'description': 'Webhook is already set'}
* Serving Flask app 'telegram_webhook_server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.24.164.227:5000
Press CTRL+C to quit

```

Figura 4.3: Bootstrap del server Flask e notifica di avvenuta registrazione del webhook.

## 4.5 Validazione Sperimentale delle Interazioni

Dopo aver avviato tutti i moduli, il chatbot è pronto su Telegram: basta cercare @MilanOfficialStoreBot. L'utente può controllare la disponibilità di un prodotto, specificando opzionalmente il target (uomo, donna, bambino). Il sistema risponde in modo preciso e suggerisce cosa fare dopo.



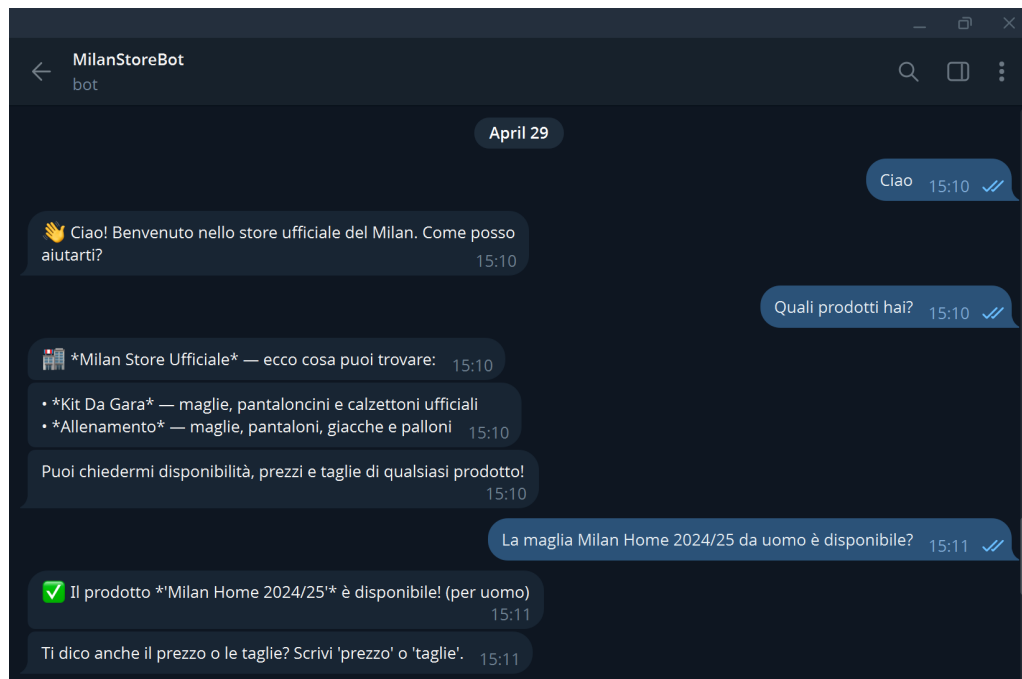


Figura 4.4: Interrogazione dello stato di disponibilità di un articolo tramite Telegram.

Dopo aver verificato la disponibilità, l'utente può chiedere il prezzo semplicemente scrivendo **prezzo**. Rasa Core mantiene il contesto della conversazione grazie alle variabili di memoria aggiornate.

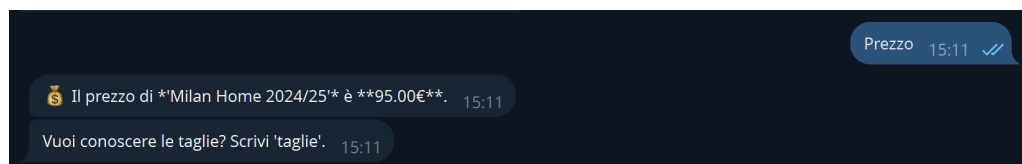


Figura 4.5: Risoluzione del prezzo di listino associato al prodotto in esame.

Allo stesso modo, l'utente può ottenere l'elenco delle taglie disponibili e i colori principali del prodotto. I suggerimenti contestuali aiutano l'utente a navigare nel catalogo senza dover ripetere il nome del prodotto.



Figura 4.6: Estrazione delle varianti di taglia e colore memorizzate nel database.

Il sistema gestisce correttamente anche i casi in cui un prodotto non è disponibile in magazzino. Se l'utente cerca il **Pallone Milan Essential** (non disponibile), il sistema lo segnala e suggerisce altre opzioni per continuare la conversazione.



Figura 4.7: Risposta del sistema per un articolo non disponibile in magazzino.

Il sistema permette di esplorare il catalogo applicando filtri per categoria o target demografico. L'utente può fare domande in linguaggio naturale, come “Che articoli ci sono per bambino?”, per vedere tutti i prodotti di quella categoria.

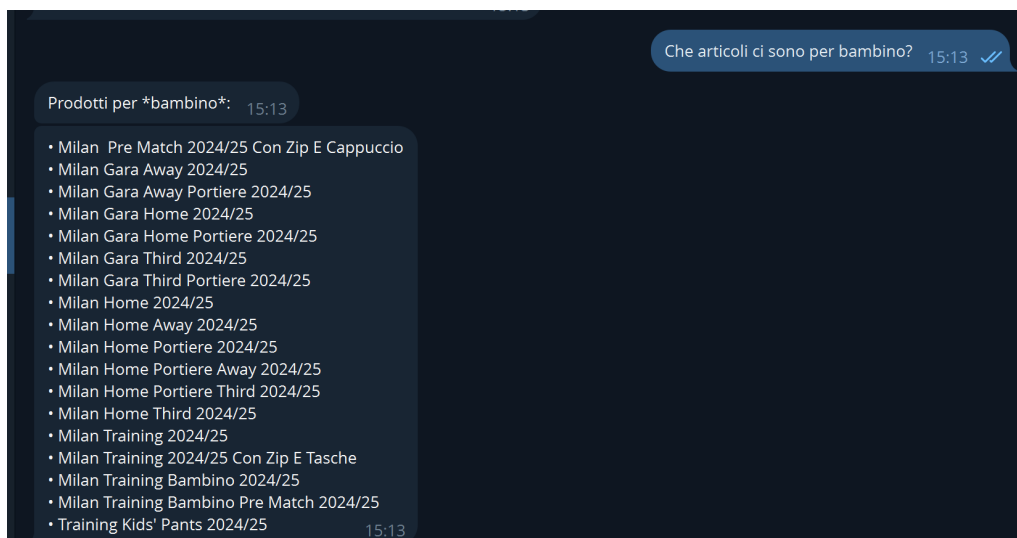


Figura 4.8: Visualizzazione dell'elenco prodotti filtrato per target demografico.

# Bibliografia

---

# Bibliografia

---

- [1] Rasa. *Documentazione ufficiale*. Disponibile al link: <https://rasa.com/docs/rasa/>
- [2] Wes McKinney. *pandas: Python Data Analysis Library*. Disponibile al link: <https://pandas.pydata.org/>
- [3] Seperman. *thefuzz: Fuzzy string matching in Python*. Disponibile al link: <https://github.com/seperman/thefuzz>
- [4] Pallets Projects. *Flask: Web Framework*. Disponibile al link: <https://flask.palletsprojects.com/>
- [5] ngrok, Inc. *ngrok: Secure tunnels to localhost*. Disponibile al link: <https://ngrok.com/>
- [6] Telegram Messenger. *Bot API*. Disponibile al link: <https://core.telegram.org/bots/api>